

Softwareentwicklung und Projektmanagement

Projekt: Numerisches Lösen

Name: Maricojevic Filip

Klasse: 3BHWI

Schuljahr: 2025/26

Gegenstand: SWP

Thema: Bisektionsverfahren, Regula falsi und Anwendung

Abgabeprotokoll

Aufgabe 5: Bisektionsverfahren

Ziel der Aufgabe

In Aufgabe 5 wurde ein Solver mit dem Bisektionsverfahren programmiert. Der Benutzer kann eine Funktion, ein Intervall und eine Genauigkeit eingeben. Das Programm nähert sich dann Schritt für Schritt einer Nullstelle an.

Funktionsweise

Beim Bisektionsverfahren wird zuerst ein Intervall $[a, b]$ gewählt. Die Funktionswerte an den beiden Grenzen müssen unterschiedliche Vorzeichen haben. Dann wird immer die Mitte des Intervalls berechnet.

Diese Mitte heißt im Programm c . Wenn $f(c)$ nahe genug bei 0 liegt, wird c als Nullstelle ausgegeben. Sonst wird entschieden, ob die Nullstelle im linken oder rechten Teilintervall liegt.

$$c = (a + b) / 2$$

Testfälle

Der Solver wurde mit der Funktion $f(x) = x^2 - n$ getestet. Dadurch wird die Quadratwurzel von n als Nullstelle berechnet.

Getestet wurden:

- $n = 25$, analytische Lösung: 5
- $n = 81$, analytische Lösung: 9
- $n = 144$, analytische Lösung: 12

```
-----  
Funktion: x**2 - n  
n = 25  
Intervall: [ 0 , 25 ]  
Epsilon = 1e-05  
Nullstelle: 4.999999701976776  
Iterationen: 24  
Abweichung f(x): 2.9802321499516893e-06  
Analytische Lösung: 5.0  
Abweichung zur Wurzel: 2.980232238769531e-07  
-----  
Funktion: x**2 - n  
n = 81  
Intervall: [ 0 , 81 ]  
Epsilon = 1e-05  
Nullstelle: 8.999999463558197  
Iterationen: 24  
Abweichung f(x): 9.655952169396187e-06  
Analytische Lösung: 9.0  
Abweichung zur Wurzel: 5.364418029785156e-07  
-----  
Funktion: x**2 - n  
n = 144  
Intervall: [ 0 , 144 ]  
Epsilon = 1e-05  
Nullstelle: 12.000000357627869  
Iterationen: 27  
Abweichung f(x): 8.583068961343088e-06  
Analytische Lösung: 12.0  
Abweichung zur Wurzel: 3.5762786865234375e-07  
  
Eigener Solver:  
Beispiel: x**2 - n  
Funktion f(x):
```

Ergebnis

Der Solver liefert für alle drei Testwerte eine Näherung, die sehr nahe an der echten Quadratwurzel liegt. Die Abweichung wird zusätzlich ausgegeben, damit die Genauigkeit überprüft werden kann.

Aufgabe 6: Regula falsi

Ziel der Aufgabe

In Aufgabe 6 wurde ein zweites Verfahren zur Nullstellensuche umgesetzt. Dafür wurde Regula falsi verwendet. Das Verfahren arbeitet auch mit einem Intervall, aber der neue Punkt wird anders berechnet als bei der Bisektion.

Unterschied zur Bisektion

Bei der Bisektion wird immer die Mitte des Intervalls genommen. Bei Regula falsi wird eine Sekante durch die Punkte $(a, f(a))$ und $(b, f(b))$ gelegt. Die Nullstelle dieser Sekante wird als neuer Näherungswert c verwendet.

$$c = b - fb * (b - a) / (fb - fa)$$

Testfälle

Regula falsi wurde wie Aufgabe 5 mit der Funktion $f(x) = x^2 - n$ getestet. Die Testwerte waren wieder $n = 25$, $n = 81$ und $n = 144$. Danach wurde das Ergebnis mit der analytischen Lösung verglichen.

```
-----
Funktion: x**2 - n
n = 25
Intervall: [ 0 , 25 ]
Epsilon = 1e-05
Nullstelle: 4.999999095622815
Iterationen: 40
Abweichung f(x): 9.043771029837444e-06
Analytische Lösung: 5.0
Abweichung zur Wurzel: 9.04377184696159e-07
-----
Funktion: x**2 - n
n = 81
Intervall: [ 0 , 81 ]
Epsilon = 1e-05
Nullstelle: 8.999999503074278
Iterationen: 78
Abweichung f(x): 8.944662752696786e-06
Analytische Lösung: 9.0
Abweichung zur Wurzel: 4.969257219045176e-07
-----
Funktion: x**2 - n
n = 144
Intervall: [ 0 , 144 ]
Epsilon = 1e-05
Nullstelle: 11.999999585723941
Iterationen: 107
Abweichung f(x): 9.942625240455527e-06
Analytische Lösung: 12.0
Abweichung zur Wurzel: 4.142760587910743e-07
```

```
Eigener Regula-falsi-Solver:
Beispiel: x**2 - n
Funktion f(x): 2*x-n
Wert für n: 10
Genauigkeit epsilon: 67
Linker Intervallwert a: 1
Rechter Intervallwert b: 10
-----
Funktion: 2*x-n
n = 10.0
Intervall: [ 1.0 , 10.0 ]
Epsilon = 67.0
Nullstelle: 5.0
Iterationen: 1
Abweichung f(x): 0.0
```

Ergebnis

Regula falsi findet ebenfalls die Nullstelle der Funktion. Je nach Funktion kann das Verfahren schneller oder langsamer sein als die Bisektion. Für dieses Projekt ist wichtig, dass beide Verfahren korrekt angewendet werden können.

Aufgabe 7: Grafische Darstellung

Ziel der Aufgabe

In Aufgabe 7 wurde die Nullstellensuche grafisch mit matplotlib dargestellt. Dadurch sieht man besser, wie sich der Solver Schritt für Schritt der Lösung nähert.

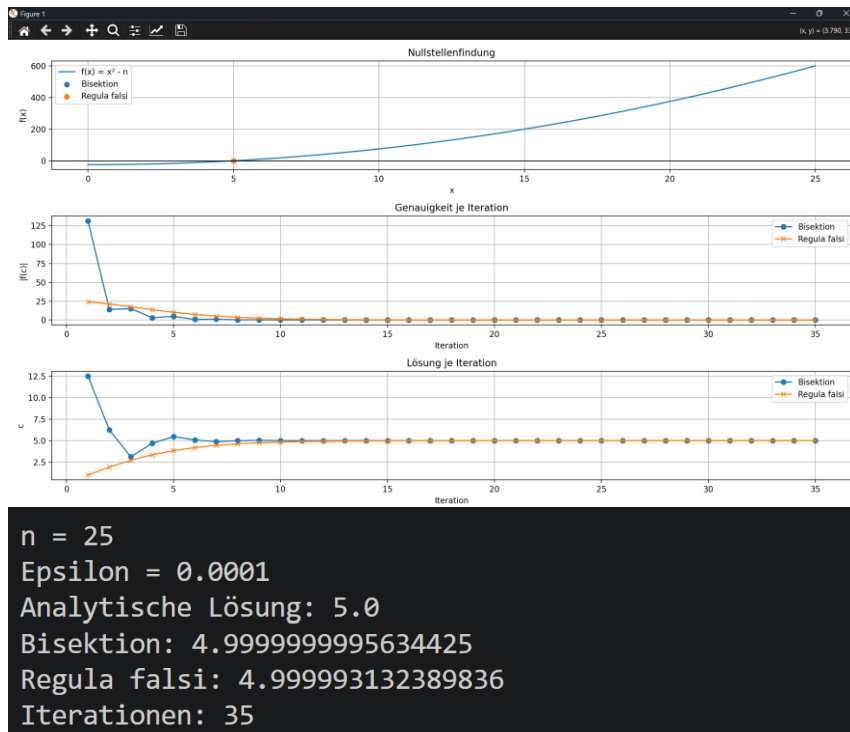
Darstellung im Programm

Das Programm zeigt drei Diagramme:

- Diagramm 1: Funktion und aktueller Näherungswert
- Diagramm 2: Genauigkeit $|f(c)|$ je Iteration
- Diagramm 3: aktuelle Lösung c je Iteration

Die Grafik wird nach jedem Iterationsschritt aktualisiert. Dafür wird im Code `plt.pause()` verwendet.

plt.pause(0.7)



Ergebnis

Die grafische Darstellung zeigt, dass die Werte mit jeder Iteration näher zur Nullstelle kommen. Bei $n = 25$ nähert sich der Wert c der Lösung 5 an.

Aufgabe 8: Polynom testen

Ziel der Aufgabe

In Aufgabe 8 wurde der Solver mit dem Polynom aus dem Theorieteil getestet. Die gesuchte Nullstelle liegt ungefähr bei $x = 3,4567$.

Polynom

$$P_4(x) = 2x + x^2 + 3x^3 - x^4$$

Intervall

Als Intervall wurde $[3, 4]$ gewählt. Dieses Intervall ist sinnvoll, weil $P_4(3)$ positiv ist und $P_4(4)$ negativ ist. Dadurch gibt es einen Vorzeichenwechsel und der Solver kann angewendet werden.

- $P_4(3) = 15$
- $P_4(4) = -40$
- Es gilt also: $P_4(3) \cdot P_4(4) < 0$

Genauigkeitstests

Die Aufgabe verlangt Tests mit $\epsilon = 10^{-2}$ und $\epsilon = 10^{-8}$. Bei einer kleineren Genauigkeit muss der Solver mehr Iterationen durchführen.

- Für $\epsilon = 10^{-2}$ werden ungefähr 11 Iterationen benötigt.
- Für $\epsilon = 10^{-8}$ werden ungefähr 30 Iterationen benötigt.
- Die berechnete Nullstelle liegt ungefähr bei $x = 3,4567$.

```
def solver() -> None:
    """Startet den Solver für Aufgabe 8."""

    try:
        funktion = "2*x + x**2 + 3*x**3 - x**4"           # Polynom aus Aufgabe 8
        n = 0                                              # n wird hier nicht benötigt
        a = 3                                              # Linke Intervallgrenze
        b = 4                                              # Rechte Intervallgrenze

        print("-----")
        print("Aufgabe 8: Polynom  $P_4(x) = 2x + x^2 + 3x^3 - x^4$ ")
        print("Passendes Intervall: [3, 4]")

        print("\nAutomatische Tests laut Angabe:")
        ausgabe(a, b, n, 10**-2, funktion, -2)           # Test mit epsilon = 10^-2
        ausgabe(a, b, n, 10**-8, funktion, -8)           # Test mit epsilon = 10^-8

        print("\nEigener Test:")
```

[illegible]

Interpretation

Man erkennt, dass ein kleineres epsilon zu mehr Iterationen führt. Das ist logisch, weil der Solver genauer rechnen muss. Die Nullstelle passt zur Angabe, daher funktioniert der Solver auch bei diesem Polynom.

Aufgabe 9: Durchhängende Leitung

Ziel der Aufgabe

In Aufgabe 9 wurde der Solver auf ein reales Problem angewendet. Es geht um eine Leitung zwischen zwei gleich hohen Masten. Die Masten sind 100 m auseinander und die Leitung hängt in der Mitte 10 m durch.

Umformung zur Nullstellenaufgabe

Zuerst wird der Krümmungsradius der Leitung gesucht. Aus der Kettenlinie ergibt sich folgende Gleichung:

$$x * \text{math.cosh}(50 / x) - x - 10 = 0$$

Hier ist x der gesuchte Krümmungsradius. Der Wert 50 wird verwendet, weil der Mast 50 m von der Mitte entfernt ist. Die 10 steht für den Durchhang.

Intervall und Berechnung

Als Intervall wurde [100, 150] verwendet. In diesem Bereich liegt der gesuchte Krümmungsradius. Der Solver aus Aufgabe 5 und das Verfahren aus Aufgabe 6 werden importiert und direkt verwendet.

- Bisektion wird aus Aufgabe 5 verwendet.
- Regula falsi wird aus Aufgabe 6 verwendet.
- Die bestehenden Funktionen werden nicht neu geschrieben, sondern importiert.

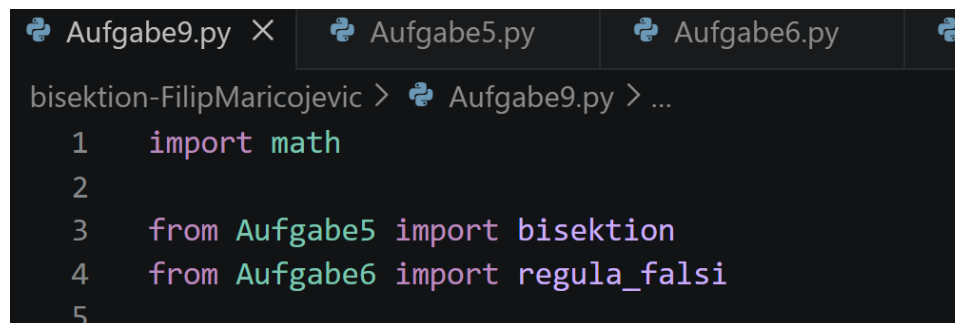
Berechnung der Leitungslänge

Nachdem der Krümmungsradius berechnet wurde, wird damit die Länge der Leitung berechnet.

$$l = 2 * \text{radius} * \text{math.sinh}(100 / (2 * \text{radius}))$$

Ergebnis

Der Krümmungsradius liegt ungefähr bei 126,63 m. Daraus ergibt sich eine Leitungslänge von ungefähr 102,6 m. Die Leitung ist also etwas länger als der direkte Abstand zwischen den Masten, weil sie durchhängt.



```
Aufgabe9.py X Aufgabe5.py Aufgabe6.py
bisektion-FilipMaricojevic > Aufgabe9.py > ...
1 import math
2
3 from Aufgabe5 import bisektion
4 from Aufgabe6 import regula_falsi
5
```

```
-----  
Aufgabe 9: Durchhängende Leitung  
Gleichung:  $x * \cosh(50 / x) - x - 10 = 0$   
Intervall: [100, 150]  
Geben Sie die Hochzahl für epsilon ein, also 10^: 10  
-----
```

```
Aufgabe 9 mit Bisektion  
Krümmungsradius: 125.0 m  
Leitungslänge: 102.68808145070388 m  
Iterationen: 1  
-----
```

```
Aufgabe 9 mit Regula falsi  
Krümmungsradius: 131.74072962757168 m  
Leitungslänge: 102.41810971029103 m  
Iterationen: 1
```